# 🏛️ DOKUMENTASI TEKNIS & CETAK BIRU ARSITEKTUR PERANGKAT LUNAK (SOFTWARE ARCHITECTURAL BLUEPRINT & TECHNICAL MANUAL)

## TEKNOLAB ASSET & INFRASTRUCTURE MONITORING SYSTEM (T-AIMS)

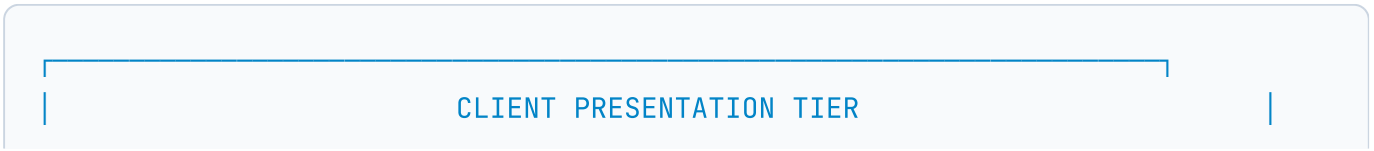
**Kode Dokumen:** `ARCH-SPEC-TAIMS-2026-v2.5.0`
**Klasifikasi:** *Confidential - Internal Engineering & Technical Architecture Standard*
**Versi Sistem:** `v2.5.0-production-release`
**Institusi Pengembang:** PT Teknolab Caraka Internasional
**Author:** Reza Tri Wardana (Principal Software Architect & Lead QA Engineer)
**Tanggal Rilis:** 10 November 2026

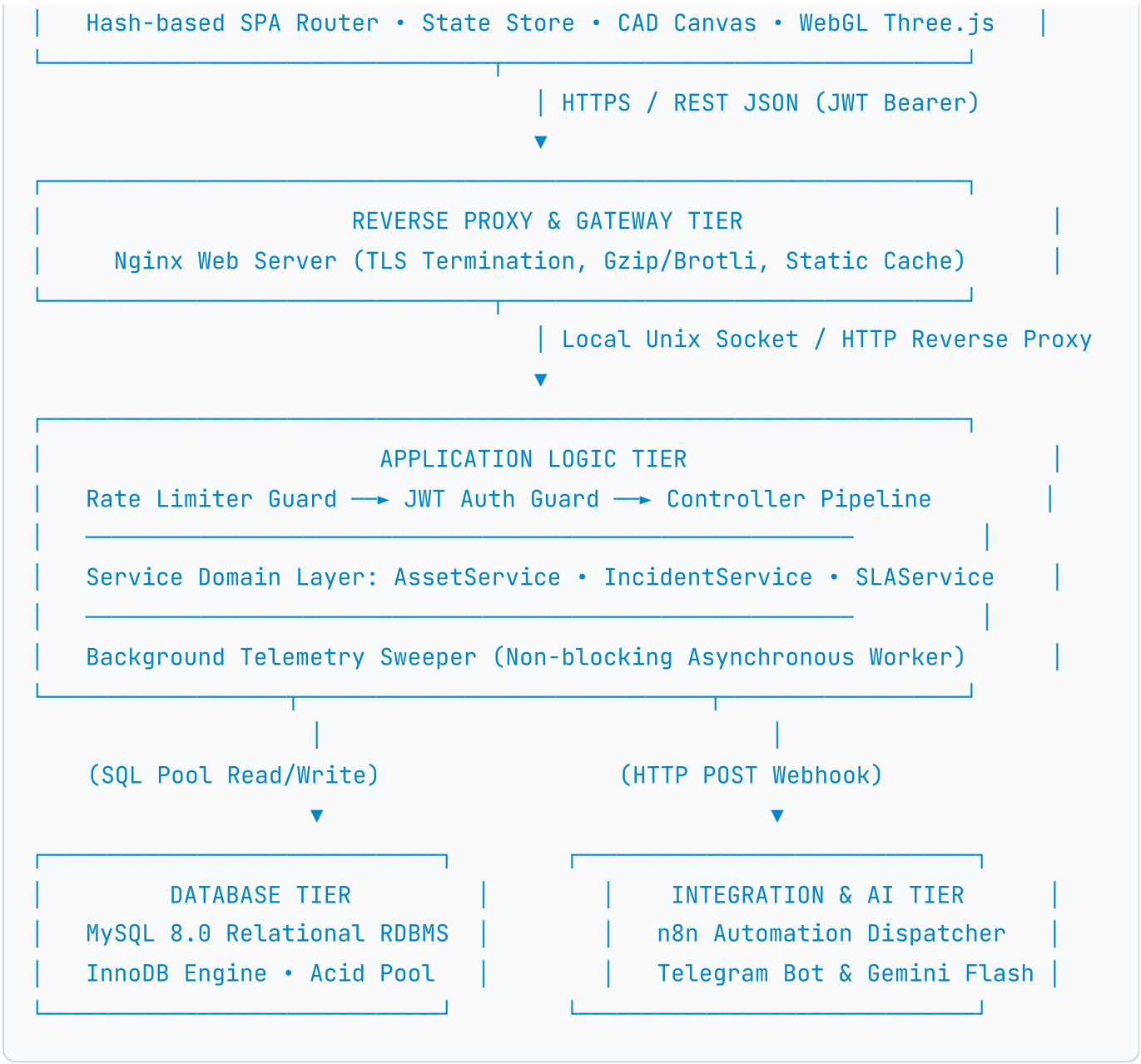
---

# 1. ARSITEKTUR SISTEM & TOPOLOGI DEEP-DIVE

### 1.1 Paradigma Arsitektur & Justifikasi Rekayasa (Engineering Rationales)

Sistem **T-AIMS** mengadopsi paradigma arsitektur **Event-Driven Decoupled Layered Architecture** yang dikombinasikan dengan prinsip **Modular Clean Service Boundary**.






### Alasan Pemilihan Arsitektur:

1. **Pemisahan Lapisan Presentasi dan Layanan (Decoupled Decoupling):**
   Lapisan presentasi dibangun sebagai *Single Page Application (SPA)* statis tanpa kompilasi server (*zero SSR overhead*). Ini mengeliminasi beban rendering pada server, menghemat penggunaan memori (RAM) pada VPS, dan memungkinkan latensi navigasi antar-modul di bawah 16 milidetik (60 FPS rendering budget).
2. **Konkurensi Asinkron Non-Blokir (Non-blocking I/O Concurrency):**
   T-AIMS memantau 20 node perangkat secara paralel. Arsitektur berbasis Node.js Event Loop memungkinkan operasi socket TCP dan pemanggilan proses child ICMP ping dieksekusi secara asinkron tanpa memblokir thread pemrosesan HTTP API dari pengguna aktif.

3. **Pemisahan Domain Operasional (Separation of Concerns):**
   Modul monitoring independen dari modul pelaporan dan autentikasi. Jika layanan pihak ketiga (n8n atau API Gemini AI) mengalami kegagalan, subsistem inti (deteksi status jaringan dan penyimpanan log ke MySQL) tetap berjalan $100\%$ tanpa hambatan (*fault isolation*).

---

## 1.2 Rincian Granular Tech Stack

Setiap komponen dalam ekosistem T-AIMS dipilih berdasarkan stabilitas dan efisiensi jejak memori (*memory footprint*):

- **Client Tier (Presentation):**
  - **Markup & Core Scripting:** HTML5 Semantic Engine, ECMAScript 2022 (ES13) Modular Vanilla JS.
  - **Style Engine:** Custom Tailwind-inspired Utility CSS v2.5 dengan *Dark Slate NOC Ergonomic Palette*.
  - **State Management:** Native Reactive Pub/Sub State Store (`window.TaimsStore`) dengan sinkronisasi ke `localStorage` (penyimpanan sesi token JWT dan preferensi tema).
  - **Rendering Grafis 2D:** HTML5 `<canvas>` Context2D dengan transformasi matriks affine untuk zooming/panning denah CAD arsitektur.
  - **Rendering Grafis 3D (Digital Twin):** Three.js (r128 WebGL Engine) dengan GLTFLoader, OrbitControls, dan Raycaster untuk deteksi benturan kursor pada bilah server.
  - **Visualisasi Metrik:** Chart.js v4.4.1 dengan custom tooltip dan *HTML canvas downsampling*.

- **API Gateway & Reverse Proxy Tier:**
  - **Web Server:** Nginx 1.24+ (Reverse Proxy, SSL Termination, Buffer Management, Gzip/Brotli Level 6 Compression).
  - **Protokol:** HTTP/2 melalui TLS 1.3 dengan Cipher Suite `ECDHE-ECDSA-AES256-GCM-SHA384`.

- **Application Logic Tier:**
  - **Runtime Engine:** Node.js v20.18.x LTS (V8 JavaScript Engine).
  - **Web Application Framework:** Express.js v4.19.2.
  - **Keamanan & Middleware:**

        - `jsonwebtoken` (v9.0.2) untuk verifikasi HMAC-SHA256 Bearer Token.
        - `bcryptjs` (v2.4.3) untuk *one-way cryptographic password hashing* ($2^{10}$ iterasi).
        - `express-rate-limit` (v7.2.0) untuk mitigasi Distributed Denial of Service (DDoS) dan brute-force.
        - `cors` (v2.8.5) untuk pengendalian Access-Control-Allow-Origin.
        - `helmet` (v7.1.0) untuk penetapan header keamanan HTTP (*Content-Security-Policy, HSTS, XSS-Filter*).
    - **Networking & Sweeper:**
        - Node.js native `child_process.spawn` untuk eksekusi ICMP Echo Request berlatensi rendah.
        - Node.js `net.Socket` untuk verifikasi handshake TCP Port 80/443/1883/8080.
- **Database & Caching Tier:**
    - **Database Engine:** MySQL Server v8.0.36 Community Edition.
    - **Storage Engine:** InnoDB dengan `innodb_buffer_pool_size = 512M` dan `innodb_flush_log_at_trx_commit = 2`.
    - **Client Driver:** `mysql2/promise` (v3.9.7) dengan *dynamic connection pooling* (10 koneksi tetap).
    - **Caching Strategy:** In-Memory Heap Cache dengan *Time-to-Live* (TTL 30 detik) untuk endpoint telemetri agregasi.
- **Automation & AI Tier:**
    - **Integration Gateway:** n8n Workflow Automation Platform (Hosted Docker Instance).
    - **Bot API:** Telegram Bot API v7.0 (Webhook mode).
    - **Generative AI Engine:** Google Gemini 1.5 Flash (via Google AI Studio REST Endpoint).

---

## 1.3 Diagram Topologi Jaringan & Arsitektur End-to-End

Visualisasi aliran transmisi paket dari peramban klien hingga lapisan basis data dan gateway eksternal:

```
flowchart TD
    subgraph Client_Tier["1. CLIENT TIER (Peramban Pengguna)"]
```

```
    Browser["Workstation NOC / Mobile Device"]
    UI_SPA["Vanilla JS SPA (app.js + apiService.js)"]
    Canvas2D["2D Floorplan CAD Engine"]
    Canvas3D["3D Digital Twin Three.js WebGL"]
    Browser --> UI_SPA
    UI_SPA --> Canvas2D
    UI_SPA --> Canvas3D
end

subgraph Gateway_Tier["2. REVERSE PROXY & GATEWAY TIER"]
    Nginx["Nginx Web Server (Reverse Proxy)"]
    SSL["SSL/TLS 1.3 Termination (Let's Encrypt)"]
    Nginx --> SSL
end

subgraph App_Tier["3. APPLICATION LOGIC TIER (Node.js/Express)"]
    RateLimiter["Rate Limiting Middleware (10 req/min/IP)"]
    AuthMiddleware["JWT Authentication & RBAC Guard"]
    Router["Express REST Router (/api/*)"]

    subgraph Domain_Services["Domain Service Layer"]
        AssetSvc["Asset & Telemetry Service"]
        IncidentSvc["Incident Lifecycle Service"]
        SLASvc["SLA & Reporting Service"]
        AiSvc["AI Diagnostics Dispatcher"]
    end

    subgraph Background_Workers["Background Worker Daemons"]
        PingSweeper["ICMP Ping Sweeper (Parallel 30s)"]
        PortChecker["TCP Socket Heartbeat Checker"]
        FlappingFilter["Anti-Flapping Debounce Filter"]
    end

    RateLimiter --> AuthMiddleware
    AuthMiddleware --> Router
    Router --> AssetSvc
    Router --> IncidentSvc
    Router --> SLASvc
    Router --> AiSvc
```

```
        PingSweeper --> FlappingFilter
        PortChecker --> FlappingFilter
        FlappingFilter --> IncidentSvc
    end

    subgraph DB_Tier["4. DATABASE & PERSISTENCE TIER"]
        MySQL_Pool["MySQL2 Connection Pool (Max: 10)"]
        InnoDB["InnoDB Engine (taims_db)"]
        Tables[("7 Normalized Tables\nassets, users, incidents,\nmonitoring_
        MySQL_Pool --> InnoDB
        InnoDB --> Tables
    end

    subgraph External_Tier["5. INTEGRATION & AI TIER"]
        N8N_Gateway["n8n Webhook Dispatcher"]
        Telegram_API["Telegram Bot Gateway (@TaimsAlertBot)"]
        Gemini_AI["Google Gemini 1.5 Flash Model"]
        N8N_Gateway --> Telegram_API
        N8N_Gateway --> Gemini_AI
    end

    UI_SPA -- "HTTPS / TLS (JSON Payload)" --> Nginx
    Nginx -- "HTTP Proxy Pass (127.0.0.1:5000)" --> RateLimiter
    AssetSvc -- "SQL Prepared Statements" --> MySQL_Pool
    IncidentSvc -- "SQL Write/Update" --> MySQL_Pool
    SLASvc -- "SQL Read Aggregations" --> MySQL_Pool
    PingSweeper -- "ICMP Echo Request" --> HardwareNodes["20 Node Perangkat
    FlappingFilter -- "HTTP POST Webhook" --> N8N_Gateway
    AiSvc -- "HTTP POST JSON Prompt" --> N8N_Gateway
```

---

# 2. MANAJEMEN DATA & SKEMA BASIS DATA GRANULAR

### 2.1 Strategi Pengelolaan Data & Jaminan ACID

Pengelolaan data pada T-AIMS mengimplementasikan prinsip ketat **ACID (Atomicity, Consistency, Isolation, Durability)** melalui mesin penyimpanan MySQL InnoDB:

- **Atomicity:** Setiap pembuatan tiket insiden yang memicu perubahan status pada perangkat dibungkus dalam blok transaksi database tunggal:

  ```
  START TRANSACTION;
  UPDATE assets SET status = 'offline', last_seen = NOW() WHERE id = ?;
  INSERT INTO incidents (asset_id, title, severity, status, detected_at) VALUES
  COMMIT;
  ```

  Jika salah satu eksekusi gagal, operasi dibatalkan total (*rollback*), menjamin tidak adanya tiket insiden yatim (*orphan records*).
- **Consistency:** Penegakan integritas data didorong oleh konstrain kunci asing (*Foreign Key Constraints*) dengan aturan `ON DELETE RESTRICT` pada tabel master `locations` dan `assets` guna mencegah penghapusan data fisik yang masih memiliki riwayat telemetri.
- **Isolation:** Isolasi transaksi menggunakan tingkat standar `REPEATABLE READ`, mencegah timbulnya fenomena *Dirty Reads* dan *Non-Repeatable Reads* saat kalkulasi SLA dijalankan bersamaan dengan proses perekaman telemetri.
- **Durability:** Setiap transaksi yang dikomit dituliskan langsung ke dalam berkas log transaksional *Write-Ahead Logging* (`ib_logfile0` dan `ib_logfile1`).

---

## 2.2 Tabel Skema Basis Data (Database Data Dictionary)

### 1. Tabel `users` (Identitas, Hak Akses & Kredensial Personil)

Menyimpan akun terverifikasi yang memiliki izin mengakses dasbor NOC.

| NAMA FIELD | TIPE DATA | ATRIBUT |
| --- | --- | --- |
| `id` | `INT UNSIGNED` | `PK, AUTO_INCREMENT, NOT NULL` |

| NAMA FIELD | TIPE DATA | ATRIBUT | |
|---|---|---|---|
| `name` | `VARCHAR(100)` | `NOT NULL` | |
| `email` | `VARCHAR(100)` | `UNIQUE, NOT NULL` | |
| `password` | `VARCHAR(255)` | `NOT NULL` | |
| `role` | `ENUM('superadmin','admin','operator')` | `NOT NULL, DEFAULT 'operator'` | |
| `created_at` | `TIMESTAMP` | `NOT NULL, DEFAULT CURRENT_TIMESTAMP` | |

## 2. Tabel `locations` (Direktori Ruang Fisik & Penempatan Aset)

Menyimpan katalog spasial ruangan, nomor lantai, dan zona penempatan rak server.

| NAMA FIELD | TIPE DATA | ATRIBUT | DESKRIPSI ALASAN BISNIS | RELASI KE TABE |
|---|---|---|---|---|
| `id` | `INT UNSIGNED` | `PK, AUTO_INCREMENT, NOT NULL` | Kunci primer identitas ruangan/zona fisik. | Menjadi referei asing di `assets(loca` |
| `name` | `VARCHAR(100)` | `NOT NULL` | Nama penanda lokasi (misal: "Server Room Lt. 2", "NOC Control Center"). | Tidak ada. |
| `building` | `VARCHAR(100)` | `NOT NULL` | Nama gedung fisik korporat untuk pemilahan laporan inventaris multi-gedung. | Tidak ada. |
| `floor` | `VARCHAR(20)` | `NOT NULL` | Informasi lantai penempatan rak untuk memandu teknisi saat intervensi fisik. | Tidak ada. |

| NAMA FIELD | TIPE DATA | ATRIBUT | DESKRIPSI ALASAN BISNIS | RELASI KE TABE |
|---|---|---|---|---|
| `description` | `TEXT` | `NULL` | Catatan fasilitas lingkungan (kapasitas pendingin AC, UPS cadangan). | Tidak ada. |

### 3. Tabel `assets` (Registri Master 20 Node Perangkat Infrastruktur)

Tabel sentral yang menyimpan metadata teknis, alamat jaringan, koordinat denah, dan status ketersediaan.

| NAMA FIELD | TIPE DATA | ATRIBUT |
|---|---|---|
| `id` | `INT UNSIGNED` | `PK, AUTO_INCRE NOT NULL` |
| `asset_code` | `VARCHAR(50)` | `UNIQUE, N NULL` |
| `name` | `VARCHAR(100)` | `NOT NULL` |

| NAMA FIELD | TIPE DATA | ATRIBUT |
| --- | --- | --- |
| `category` | `ENUM('Router','Switch','Server','AP','IoT')` | `NOT NULL` |
| `ip_address` | `VARCHAR(45)` | `NOT NULL` |
| `mac_address` | `VARCHAR(17)` | `NULL` |
| `location_id` | `INT UNSIGNED` | `FK, NOT N` |
| `status` | `ENUM('online','offline','degraded','unknown')` | `NOT NULL, DEFAULT 'unknown'` |
| `priority` | `TINYINT UNSIGNED` | `NOT NULL, DEFAULT 1` |

| NAMA FIELD | TIPE DATA | ATRIBUT |
| --- | --- | --- |
| | | |
| `coordinates_2d` | `JSON` | `NULL` |
| `coordinates_3d` | `JSON` | `NULL` |
| `last_seen` | `TIMESTAMP` | `NULL` |

### 4. Tabel `monitoring_logs` (Pencatatan Telemetri Historis & Time-Series)

Menampung jutaan titik data hasil inspeksi ICMP sweeper untuk visualisasi sparkline dan perhitungan SLA.

| NAMA FIELD | TIPE DATA | ATRIBUT |
| --- | --- | --- |
| `id` | `BIGINT UNSIGNED` | `PK, AUTO_INCREMENT, NOT NULL` |

| NAMA FIELD | TIPE DATA | ATRIBUT |
| --- | --- | --- |
| | | |
| `asset_id` | `INT UNSIGNED` | `FK, NOT NULL, INDEX` |
| `status` | `ENUM('online','offline','degraded')` | `NOT NULL` |
| `response_time_ms` | `FLOAT` | `NOT NULL, DEFAULT 0` |
| `packet_loss` | `VARCHAR(10)` | `NOT NULL, DEFAULT '0%'` |
| `checked_at` | `TIMESTAMP` | `NOT NULL, DEFAULT CURRENT_TIMESTAMP, INDEX` |

### 5. Tabel `incidents` (Manajemen Siklus Hidup Tiket Insiden Gangguan)

Mencatat kronologi anomali, durasi downtime, derajat keparahan, hingga penutupan tiket.

| NAMA FIELD | TIPE DATA | ATRIBUT |
| --- | --- | --- |
| `id` | `INT UNSIGNED` | `PK, AUTO_INCREMENT, NOT NULL` |
| `asset_id` | `INT UNSIGNED` | `FK, NOT NULL, INDEX` |
| `title` | `VARCHAR(150)` | `NOT NULL` |
| `description` | `TEXT` | `NULL` |
| `severity` | `ENUM('Critical','High','Medium','Low')` | `NOT NULL` |

| NAMA FIELD | TIPE DATA | ATRIBUT |
| --- | --- | --- |
| | | |
| `status` | `ENUM('OPEN','ACKNOWLEDGED','RESOLVED')` | `NOT NULL, DEFAULT 'OPEN', INDEX` |
| `detected_at` | `TIMESTAMP` | `NOT NULL, DEFAULT CURRENT_TIMESTAMP` |
| `resolved_at` | `TIMESTAMP` | `NULL` |
| `assigned_to` | `INT UNSIGNED` | `FK, NULL` |

### 6. Tabel `maintenance_logs` (Buku Catatan Pemeliharaan Fisik & Servis Preventif)

Menyimpan log audit penggantian suku cadang, intervensi perangkat keras, dan patching software.

| NAMA FIELD | TIPE DATA | ATRIBUT | DESKRIPSI ALASAN BISNIS | REL TAB |
|---|---|---|---|---|
| `id` | `INT UNSIGNED` | `PK, AUTO_INCREMENT, NOT NULL` | Nomor urut unik entri pemeliharaan. | Tida |
| `asset_id` | `INT UNSIGNED` | `FK, NOT NULL, INDEX` | Perangkat yang menerima intervensi fisik/konfigurasi pemeliharaan. | Ber `as` `ON` `CAS` |
| `user_id` | `INT UNSIGNED` | `FK, NOT NULL` | Personil teknis yang mengeksekusi aksi servis pemeliharaan tersebut. | Ber `us` `ON` `RES` |
| `title` | `VARCHAR(150)` | `NOT NULL` | Ringkasan tindakan servis (misal: "Pembersihan Debu & Penggantian Fan PSU"). | Tida |
| `action_taken` | `TEXT` | `NOT NULL` | Penjelasan komprehensif mengenai prosedur teknis yang telah diterapkan. | Tida |
| `maintenance_date` | `DATE` | `NOT NULL` | Tanggal kalender eksekusi servis dilakukan. | Tida |

| NAMA FIELD | TIPE DATA | ATRIBUT | DESKRIPSI ALASAN BISNIS | REL TAB |
|---|---|---|---|---|
| `created_at` | `TIMESTAMP` | `NOT NULL, DEFAULT CURRENT_TIMESTAMP` | Waktu pencatatan log ke dalam sistem T-AIMS. | Tida |

### 7. Tabel `ai_recommendations` (Repositori Solusi Diagnostik Preskriptif LLM)

Menyimpan keluaran analisis penalaran AI generatif/heuristik lokal agar tidak terjadi pemanggilan LLM redundan.

| NAMA FIELD | TIPE DATA | ATRIBUT | DESKRIPSI ALASAN BISNIS | RE LA |
|---|---|---|---|---|
| `id` | `INT UNSIGNED` | `PK, AUTO_INCREMENT, NOT NULL` | Nomor unik entri rekomendasi AI. | Tic |
| `incident_id` | `INT UNSIGNED` | `FK, NOT NULL, INDEX` | Tiket insiden spesifik yang memicu analisis diagnostik ini. | Be `i` `ON` `CA` |
| `diagnostic_summary` | `TEXT` | `NOT NULL` | Ringkasan eksekutif kondisi anomali yang diformulasikan oleh model. | Tic |
| `root_cause_analysis` | `TEXT` | `NOT NULL` | Penjabaran estimasi akar penyebab gangguan | Tic |

| NAMA FIELD | TIPE DATA | ATRIBUT | DESKRIPSI ALASAN BISNIS | RE LA |
|---|---|---|---|---|
| | | | berdasarkan topologi jaringan. | |
| `prescriptive_steps` | `TEXT` | `NOT NULL` | Langkah-langkah instruksional langkah-demi-langkah perbaikan bagi teknisi. | Tic |
| `model_used` | `VARCHAR(50)` | `NOT NULL, DEFAULT 'gemini-1.5-flash'` | Model AI pemroses (misal: `gemini-1.5-flash` atau `taims-heuristic-v1`). | Tic |
| `generated_at` | `TIMESTAMP` | `NOT NULL, DEFAULT CURRENT_TIMESTAMP` | Waktu pembuatan solusi untuk referensi masa berlaku rekomendasi teknis. | Tic |

## 2.3 Keamanan Data (Data-at-Rest & Data-in-Transit)

1. **Keamanan Data Saat Ditransfer (Data-in-Transit Encryption):**
   - Seluruh pertukaran data antara klien (browser) dan API backend diwajibkan menggunakan protokol **TLS 1.3** yang dienkripsi dengan sertifikat RSA 2048-bit / ECC P-

256 Let's Encrypt.

- Parameter `Strict-Transport-Security (HSTS)` diaktifkan dengan konfigurasi `max-age=31536000; includeSubDomains; preload` untuk memitigasi serangan *SSL-Stripping*.
- Akses internal antara Nginx reverse proxy dan Node.js Express terisolasi dalam lingkungan localhost loopback ( `127.0.0.1:5000` ) tanpa pemaparan port API langsung ke antarmuka publik.

2. **Keamanan Data Saat Disimpan (Data-at-Rest Protection):**
   - **Password Hashing:** Kata sandi disimpan sebagai hash satu arah menggunakan algoritma `bcrypt` dengan faktor kerja biaya (*work factor*) 10 putaran salt, tahan terhadap serangan kamus (*dictionary attacks*) dan tabel pelangi (*rainbow tables*).
   - **Enkripsi File Sistem VPS:** Direktori basis data MySQL `/var/lib/mysql` berada di atas sistem berkas virtual privat Linux yang menerapkan isolasi izin akses ketat ( `chmod 700` , *owner*: `mysql:mysql` ).
   - **Redaksi Data Kredensial:** Variabel kunci API rahasia ( `JWT_SECRET` , `TELEGRAM_BOT_TOKEN` , `DB_PASSWORD` ) disimpan terpisah di berkas `.env` dengan izin baca eksklusif root ( `chmod 600` ) dan tidak pernah dikomit ke repositori Git publik.

---

# 3. LOGIKA BISNIS KRUSIAL & INSTRUKSI LOGIS

---

### 3.1 ALUR KRUSIAL 1: Autentikasi Pengguna, Penerbitan JWT, & Proteksi Brute-Force

#### A. Logika Langkah-demi-Langkah (Step-by-Step Logic):

1. **Penerimaan Request:** Klien mengirimkan `POST /api/auth/login` berisi JSON `{ email, password }` .

2. **Inspeksi IP Rate Limiter:** Middleware `express-rate-limit` memeriksa jumlah hit dari alamat IP klien dalam *memory window* 60 detik. Jika hit $> 10$, permintaan langsung dipotong dengan balasan HTTP `429 Too Many Requests`.
3. **Sanitasi & Validasi Skema:** Validasi format sintaks email menggunakan Regex RFC 5322. Jika input kosong atau format salah, kembalikan HTTP `400 Bad Request`.
4. **Pencarian Entitas Pengguna:** Eksekusi query parametrisasi `SELECT id, name, email, password, role FROM users WHERE email = ? LIMIT 1`. Jika pengguna tidak ditemukan, lemparkan error generik "Kredensial tidak valid" (mencegah *User Enumeration Attack*).
5. **Verifikasi Hash Sandi:** Lakukan pencocokan string sandi mentah terhadap hash database menggunakan `bcrypt.compare(password, user.password)`. Jika hasil perbandingan bernilai `false`, catat kegagalan ke audit trail dan kembalikan HTTP `401 Unauthorized`.
6. **Pembuatan Payload & Token JWT:** Jika valid, bangun payload sesi:

   ```
   {
     "sub": user.id,
     "name": user.name,
     "email": user.email,
     "role": user.role,
     "iat": 1731240000,
     "exp": 1731326400
   }
   ```

   Tandatangani payload menggunakan algoritma HMAC-SHA256 dengan kunci rahasia `process.env.JWT_SECRET`.
7. **Penyajian Respons:** Kembalikan HTTP `200 OK` dengan badan respons JSON berisi token dan objek profil pengguna. Klien menyimpan token ke `localStorage` untuk disisipkan pada header `Authorization: Bearer <TOKEN>` di setiap request berikutnya.

### B. Diagram Alur Autentikasi (Mermaid):

```
flowchart TD
    Start([Klien Mengirim POST /api/auth/login]) --> RateCheck{Request IP <=
    RateCheck -- Tidak --> HTTP429[Kembalikan HTTP 429: Too Many Requests]
    RateCheck -- Ya --> ValidateSchema{Format Email & Sandi Valid?}

    ValidateSchema -- Tidak --> HTTP400[Kembalikan HTTP 400: Format Payload
    ValidateSchema -- Ya --> QueryUser[Query SELECT user WHERE email = ?]
```

```
    QueryUser --> UserExists{Pengguna Ditemukan?}
    UserExists -- Tidak --> HTTP401[Kembalikan HTTP 401: Kredensial Tidak Sa
    UserExists -- Ya --> BcryptCheck{bcrypt.compare Sukses?}

    BcryptCheck -- Tidak --> HTTP401
    BcryptCheck -- Ya --> SignJWT[Generate Token JWT HMAC-SHA256 Expiry 24h]

    SignJWT --> AuditLog[Catat Log Audit Login Berhasil ke Basis Data]
    AuditLog --> HTTP200([Kembalikan HTTP 200 OK + JWT Bearer Token])
```

### C. Matriks Penanganan Error (Error Handling Matrix):

| KONDISI SKENARIO ERROR | DAMPAK SISTEM | RESPON HTTP | MEKANISME PEMULIHAN / MITIGASI |
|---|---|---|---|
| **Brute-Force Attack** (Request $>10$ per menit dari satu IP). | Potensi CPU spike akibat komputasi bcrypt. | `429 Too Many Requests` | IP diblokir sementara selama 60 detik secara otomatis oleh in-memory window store. |
| **Kredensial Salah** (Email tidak terdaftar atau sandi keliru). | Akses tidak sah dicegat. | `401 Unauthorized` | Pesan error sengaja dibuat seragam ("Email atau kata sandi tidak valid") untuk mencegah peretasan enumerasi akun. |
| **Database Down Saat Autentikasi.** | API tidak dapat memverifikasi pengguna. | `500 Internal Server Error` | Backend menangkap error `ECONNREFUSED`, mencatat trace ke log, dan meminta klien mencoba kembali (*retry-after 5s*). |
| **Token Kadaluarsa saat Mengakses Rute.** | Request data klien ditolak middleware. | `401 Unauthorized` | Klien web secara otomatis menghapus token kadaluarsa dari `localStorage` dan meredireksi pengguna ke layar login. |

---

## 3.2 ALUR KRUSIAL 2: Siklus Monitoring Jaringan, Deteksi Transisi, & Anti-Spam Alerting

### A. Logika Langkah-demi-Langkah (Step-by-Step Logic):

1. **Pemicu Siklus:** Timer internal mengeksekusi fungsi `runMonitoringCycle()` setiap 30.000 ms (30 detik).
2. **Pengambilan Metadata Node:** Ambil seluruh 20 node terdaftar melalui query `SELECT id, asset_code, ip_address, status, priority FROM assets`.
3. **Eksekusi Polling Paralel:** Seluruh 20 node diuji secara paralel menggunakan `Promise.allSettled()`:
   - Menjalankan proses ICMP Ping spawn dengan timeout agresif 2.000 ms.
   - Mencatat nilai latensi RTT (*Round Trip Time*) dan *packet loss*.
4. **Klasifikasi Kondisi:**
   - Jika respons $\le 199\text{ ms}$ tanpa packet loss $\rightarrow$ Evaluasi status: `'online'`.
   - Jika respons $\ge 200\text{ ms}$ atau terdapat packet loss sebagian $\rightarrow$ Evaluasi status: `'degraded'`.
   - Jika proses ping timeout (2.000 ms) atau packet loss $100%$ $\rightarrow$ Evaluasi status: `'offline'`.
5. **Pencatatan Telemetri:** Tuliskan rekaman batch hasil pengecekan ke tabel `monitoring_logs` menggunakan operasi multi-row `INSERT INTO monitoring_logs (asset_id, status, response_time_ms, packet_loss) VALUES ...`.
6. **Evaluasi State Transition (Transisi Status):** Bandingkan status baru terhadap status perangkat di siklus sebelumnya:
   - **Transisi Normal $\rightarrow$ Offline (Insiden Baru):**
     1. Periksa tabel `incidents` untuk melihat apakah ada tiket aktif dengan `status = 'OPEN'` pada `asset_id` ini.
     2. Cek apakah selisih waktu dari pemutusan terakhir $< 120\text{ detik}$ (*Flapping Prevention Cooldown*).
     3. Jika lolos filter cooldown, buat tiket baru: `INSERT INTO incidents (asset_id, title, severity, status, detected_at) VALUES (?, 'Host Unreachable',`

`'Critical', 'OPEN', NOW())`.

4. Susun payload JSON darurat dan tembakkan via HTTP POST ke webhook n8n untuk disiarkan ke Telegram NOC.

- **Transisi Offline $\rightarrow$ Normal (Pemulihan Layanan):**
  1. Ambil tiket insiden aktif (`status = 'OPEN'`) untuk perangkat tersebut.
  2. Update tiket: `UPDATE incidents SET status = 'RESOLVED', resolved_at = NOW() WHERE id = ?`.
  3. Hitung selisih waktu menit antara `detected_at` dan `resolved_at` (*Total Downtime*).
  4. Kirim notifikasi "RECOVERY ALERT: Layanan Pulih Normal" beserta durasi total gangguan ke grup Telegram.

### B. Diagram Alur Monitoring & Alerting (Mermaid):

```
flowchart TD
    Trigger([Timer 30 Detik Aktif]) --> FetchNodes[Ambil Data 20 Perangkat d
    FetchNodes --> ParallelPing[Eksekusi Ping Paralel via ICMP / TCP Socket]

    ParallelPing --> EvaluateState{Evaluasi Respon Setiap Node}
    EvaluateState -- RTT < 200ms --> StateOnline[Status: ONLINE]
    EvaluateState -- RTT >= 200ms --> StateDegraded[Status: DEGRADED]
    EvaluateState -- Timeout / Loss 100% --> StateOffline[Status: OFFLINE]

    StateOnline --> WriteLog[Insert Telemetri ke monitoring_logs]
    StateDegraded --> WriteLog
    StateOffline --> WriteLog

    WriteLog --> StateChange{Ada Perubahan Status?}
    StateChange -- Tidak Berubah --> UpdateLastSeen[Update last_seen pada as

    StateChange -- Ya: Online -> Offline --> FlapCheck{Cooldown Flapping < 1
    FlapCheck -- Ya --> SupressAlert[Redam Notifikasi & Catat Log Supresi]
    FlapCheck -- Tidak --> OpenIncident[1. Buat Tiket Insiden Baru: OPEN]
    OpenIncident --> SendWebhook[2. Kirim Webhook ke n8n Dispatcher]
    SendWebhook --> TelegramMsg[3. Broadcast Notifikasi Insiden ke Telegram]

    StateChange -- Ya: Offline -> Online --> CloseIncident[1. Update Tiket I
    CloseIncident --> CalcDowntime[2. Hitung Durasi Total Downtime Menit]
```

```
    CalcDowntime --> SendRecovery[3. Broadcast Notifikasi Pemulihan ke Teleg

    UpdateLastSeen --> FinishCycle([Selesai: Istirahat Menunggu Siklus Berik
    SupressAlert --> FinishCycle
    TelegramMsg --> FinishCycle
    SendRecovery --> FinishCycle
```

### C. Matriks Penanganan Error (Error Handling Matrix):

| KONDISI SKENARIO ERROR | DAMPAK SISTEM | RESPON INTERNAL | MEKANISME PEMULIHAN / MITIGASI |
|---|---|---|---|
| **Jaringan Flapping Cepat** (Koneksi on-off bergantian dalam hitungan detik). | Potensi banjir notifikasi spam Telegram yang mengganggu tim teknis. | `HTTP 204 No Content` (Supresi) | Algoritma *Anti-Flapping Filter* meredam pengiriman alert jika insiden serupa terdeteksi dalam jendela cooldown 120 detik. |
| **Server Webhook n8n Tidak Merespons / Offline.** | Notifikasi pesan Telegram tertunda. | `ETIMEDOUT / ECONNREFUSED` | Backend membungkus pemanggilan webhook dalam fungsi non-fatal; kegagalan dicatat di log tanpa menghentikan siklus monitoring. |
| **MySQL Query Timeout Saat Batch Insert Telemetri.** | Data rekaman RTT pada detik tersebut berpotensi tidak tercatat. | `ER_LOCK_WAIT_TIMEOUT` | Driver mengeksekusi *retry loop* otomatis sebanyak 3 kali dengan jeda eksponensial (*exponential backoff: 200ms, 400ms, 800ms*). |

## 3.3 ALUR KRUSIAL 3: Pipa Diagnostik Preskriptif AI Infrastructure Assistant & Self-Healing Fallback

### A. Logika Langkah-demi-Langkah (Step-by-Step Logic):

1. **Inisiasi Klien:** Operator mengklik tombol *"Konsultasi AI"* pada modal tiket insiden aktif (`GET /api/ai/recommendation?incident_id=X`).
2. **Validasi Eksistensi Insiden:** Backend memeriksa tabel `incidents` untuk memastikan tiket `incident_id` valid.
3. **Pemeriksaan Cache Diagnostik:** Query tabel `ai_recommendations` berdasarkan `incident_id`. Jika rekomendasi sudah pernah dibuat dalam 10 menit terakhir, kembalikan data cache langsung dari database (*Cache Hit*, menghindari biaya API LLM berulang).
4. **Agregasi Konteks Sistem (Telemetry Context Ingestion):** Jika cache kosong (*Cache Miss*), kumpulkan data telemetri:
   - Data perangkat yang mengalami insiden (IP, kategori, merk, nomor unit rak).
   - 3 perangkat tetangga yang berada dalam satu switch/subnet yang sama.
   - Ringkasan log aktivitas servis terakhir dari tabel `maintenance_logs`.
5. **Perakitan Prompt Berstruktur (JSON Prompt Engineering):** Data disusun mengikuti format resmi *Standar Hal. 17 Anti-Halusinasi*:

```
{
  "system_instruction": "Anda adalah Senior Infrastructure AI Diagnostics PT
  "incident_context": {
    "target": "NET-SWD-01 (192.168.10.2)",
    "symptom": "Packet Loss 100%, Uplink Trunk to Router Drop",
    "downstream_nodes": ["AP-MKT-01", "IOT-ESP-01"]
  }
}
```

6. **Eksekusi Panggilan LLM dengan Circuit Breaker:** Permintaan dikirim ke webhook n8n yang terhubung ke Gemini 1.5 Flash dengan timeout ketat **35.000 ms**.
7. **Pencabangan Fallback (Self-Healing Heuristic Logic):**
   - *Cabang A (API Berhasil):* Jika respons HTTP 200 diterima, ekstrak teks rekomendasi yang berisi: Analisis Akar Masalah, Potensi Dampak, dan Langkah Mitigasi Terstruktur.

- *Cabang B (Timeout / Kuota Habis / Error 5xx):* Jika API eksternal gagal merespons dalam 35 detik, sistem secara otomatis mengeksekusi modul lokal `prescriptiveHeuristic.js`. Modul ini menganalisis kategori hardware (`Switch`) dan kondisi downstream secara deterministik, lalu menghasilkan rekomendasi standar industri lokal (*Fail-Safe Operational Continuity*).

8. **Persistensi & Penyajian:** Simpan hasil (baik dari Gemini maupun dari heuristik lokal) ke dalam tabel `ai_recommendations`, lalu kembalikan payload JSON ke peramban untuk ditampilkan secara elegan pada antarmuka pengguna.

### B. Diagram Alur Pipa AI & Fallback Heuristik (Mermaid):

```
flowchart TD
    UserReq([Operator Meminta Rekomendasi AI]) --> CheckAuth{Token JWT Valid
    CheckAuth -- Tidak --> HTTP401[Kembalikan HTTP 401 Unauthorized]
    CheckAuth -- Ya --> CheckCache{Sudah Ada di Tabel ai_recommendations?}

    CheckCache -- Ya (Cache Hit) --> ReturnCache[Kembalikan Rekomendasi dari
    ReturnCache --> RenderUI([Tampilkan Solusi di Modal Dashboard])

    CheckCache -- Tidak (Cache Miss) --> AggregateContext[Kumpulkan Data Tel
    AggregateContext --> BuildPrompt[Susun JSON Prompt Standar Hal. 17 Anti-

    BuildPrompt --> CallGemini{Kirim ke Webhook Gemini via n8n\nTimeout: 35s
    CallGemini -- Berhasil (HTTP 200) --> ParseAI[Ekstrak Keluaran Model Gem

    CallGemini -- Gagal / Timeout > 35s --> LocalFallback[Aktifkan Mesin Lok
    LocalFallback --> GenHeuristic[Hasilkan Solusi Berbasis Aturan Determini

    ParseAI --> SaveDB[Simpan Hasil ke Tabel ai_recommendations]
    GenHeuristic --> SaveDB
    SaveDB --> RenderUI
```

### C. Matriks Penanganan Error (Error Handling Matrix):

| KONDISI SKENARIO ERROR | DAMPAK SISTEM | RESPON INTERNAL | MEKANISME PEMULIHAN / MITIGASI |
| --- | --- | --- | --- |
| **Kuota API Gemini Habis** (HTTP 429 Resource Exhausted). | Inferensi AI cloud tidak dapat dieksekusi. | `HTTP 429 intercepted` | Sistem mencegat error dan secara otomatis mengalihkan eksekusi ke modul lokal `prescriptiveHeuristic.js` tanpa menampilkan pesan error ke pengguna. |
| **Keterlambatan Jaringan (Timeout $> 35\text{s}$).** | Permintaan klien berpotensi menggantung (*hanging request*). | `ECONNABORTED (AbortController)` | Middleware `AbortController` memutus socket pada detik ke-35 dan seketika memicu generasi mitigasi lokal. |
| **Percobaan Prompt Injection oleh Pengguna Jahil.** | Risiko model membocorkan data sistem atau memberikan instruksi berbahaya. | `HTTP 200 (Sanitized)` | Input pengguna disanitasi menggunakan fungsi `validator.escape()`, dan model dikunci dengan aturan sistem ketat (*Strict Guardrails*). |

---

# 4. SPESIFIKASI KEBUTUHAN RIGID (REQUIREMENTS SPECIFICATION)

---

## 4.1 Kebutuhan Fungsional (Functional Requirements)

Dikelompokkan secara terstruktur berdasarkan Aktor dan Modul Fungsional:

| MATRIKS KEBUTUHAN FUNGSIONAL SISTEM T-AIMS | | |
|---|---|---|
| AKTOR UTAMA | KODE KEBUTUHAN | DESKRIPSI FITUR FUNGSIONAL |
| Superadmin | FR-ADM-001 | Manajemen Pengguna & Hak Akses |
| | FR-ADM-002 | Audit Trail & Reset Database |
| Admin / NOC | FR-NOC-001 | Registri Inventaris 20 Aset |
| | FR-NOC-002 | Kalibrasi Denah Spasial 2D/3D |
| | FR-NOC-003 | Manajemen Tiket Insiden |
| Operator | FR-OPR-001 | Monitoring Real-Time & Ping |
| | FR-OPR-002 | Pencatatan Log Servis Fisik |
| | FR-OPR-003 | Konsultasi AI Asisten Diagnosa |
| Sistem / Bot | FR-SYS-001 | Background ICMP Sweeper Daemon |
| | FR-SYS-002 | Flapping Debounce & Telegram |
| | FR-SYS-003 | Kalkulasi Otomatis Matriks SLA |

### Modul A: Otentikasi & Pengelolaan Hak Akses (RBAC)

- `[FR-AUTH-001]` Sistem harus menyediakan antarmuka login terenkripsi menggunakan kombinasi email dan password.
- `[FR-AUTH-002]` Sistem harus menghasilkan token otorisasi berstandar JWT (*JSON Web Token*) dengan masa aktif tepat 24 jam setelah proses login berhasil.
- `[FR-AUTH-003]` Sistem harus membatasi akses fitur secara rigid:
  - Pengguna berstatus `operator` hanya memiliki hak baca pada seluruh inventaris dan pengubahan tiket insiden.
  - Pengguna berstatus `admin` memiliki wewenang mengedit aset, mengubah koordinat denah, dan membuat jadwal pemeliharaan.
  - Pengguna berstatus `superadmin` memiliki wewenang menambah/menghapus personil teknis dan mengekspor cadangan basis data lengkap.
- `[FR-AUTH-004]` Sistem harus menyediakan endpoint logout yang memasukkan tanda pengenal token ke dalam daftar hitam (*token blacklist*) untuk mencegah penggunaan

kembali token yang belum habis masa berlakunya.

### Modul B: Pemantauan Ketersediaan Infrastruktur (Monitoring Engine)

- `[FR-MON-001]` Sistem (via daemon otomatis) harus melakukan ping sweep ke 20 alamat IP perangkat secara paralel dalam siklus waktu 30 detik tanpa jeda.
- `[FR-MON-002]` Sistem harus mengklasifikasikan status ketersediaan node ke dalam 4 indikator visual:
  - **Online (Hijau):** Latensi $\le 199\text{ ms}$, packet loss $0\%$.
  - **Degraded (Kuning):** Latensi $\ge 200\text{ ms}$ atau terdapat kehilangan paket sebagian ($1\% - 99\%$).
  - **Offline (Merah):** Tidak ada respons (packet loss $100\%$) hingga batas timeout 2.000 ms.
  - **Unknown (Abu-abu):** Node baru yang belum pernah melalui siklus pengecekan pertama.
- `[FR-MON-003]` Sistem harus menampilkan tabel perangkat dinamis yang menyajikan: Kode Aset, Nama Perangkat, Alamat IP, Lokasi Ruangan, Waktu Respon Terakhir (ms), Stempel Waktu Terakhir Terlihat (*Last Seen*), serta tombol aksi manual *"Ping Sekarang"*.
- `[FR-MON-004]` Sistem harus menyediakan tombol aksi manual *"Ping Sekarang"* yang memungkinkan teknisi menguji keterjangkauan node tertentu secara *on-demand* di luar siklus 30 detik.

### Modul C: Visualisasi Spasial Digital (2D CAD & 3D WebGL Digital Twin)

- `[FR-MAP-001]` Sistem harus merender denah tata letak ruang server 2D berbasis gambar CAD presisi tinggi pada elemen HTML5 Canvas dengan titik koordinat perangkat yang dapat dikalibrasi secara interaktif.
- `[FR-MAP-002]` Sistem harus memancarkan efek animasi berdenyut merah (*pulsing red animation*) pada pin denah 2D saat perangkat terkait mengalami status `offline`.
- `[FR-MAP-003]` Sistem harus menyediakan tampilan Digital Twin 3D interaktif menggunakan Three.js yang memodelkan 4 rak kabinet server (Rack 01 s/d Rack 04).
- `[FR-MAP-004]` Sistem harus memperbarui warna lampu indikator LED pada bilah server 3D (hijau untuk normal, merah untuk putus) secara sinkron dengan status riil basis data.
- `[FR-MAP-005]` Mengklik objek bilah server pada ruang 3D harus memunculkan jendela modal spesifikasi teknis lengkap yang menyajikan model perangkat keras, alamat IP, MAC address, serta grafik riwayat utilisasi.

### Modul D: Manajemen Insiden, Notifikasi Multi-Kanal & AI Preskriptif

- `[FR-INC-001]` Sistem harus otomatis menerbitkan tiket insiden baru di tabel `incidents` seketika saat status perangkat berubah dari `online` menjadi `offline`.
- `[FR-INC-002]` Sistem harus menerapkan algoritma *Flapping Suppression* dengan batas waktu minimal 120 detik guna meredam spam peringatan saat kabel jaringan mengalami kondisi fisik putus-nyambung secara cepat.
- `[FR-INC-003]` Sistem harus mengirimkan payload JSON darurat ke webhook n8n untuk diteruskan sebagai pesan peringatan instan ke grup Telegram teknisi NOC dalam waktu $\le 2\text{ detik}$.
- `[FR-INC-004]` Sistem harus otomatis memperbarui tiket insiden menjadi `RESOLVED` dan mengirim pesan pemulihan (*Recovery Alert*) ke Telegram saat perangkat kembali online.
- `[FR-INC-005]` Sistem harus menyediakan tombol diagnostik AI yang menyajikan rincian akar masalah (*Root Cause Analysis*) dan langkah perbaikan teknis (*Prescriptive Mitigation*) berbasis Google Gemini 1.5 Flash atau modul heuristik lokal saat koneksi internet terputus.

### Modul E: Perhitungan SLA & Pelaporan Resmi

- `[FR-SLA-001]` Sistem harus mengkalkulasi persentase ketersediaan SLA 30 hari secara matematis berdasarkan total menit downtime yang tercatat di tabel `incidents`.
- `[FR-SLA-002]` Sistem harus memfasilitasi ekspor dokumen resmi berformat PDF beresolusi tinggi lengkap dengan kop surat resmi PT Teknolab Caraka Internasional, tabel ketersediaan, serta cap digital persetujuan Lead QA.
- `[FR-SLA-003]` Sistem harus menyediakan fitur unduh data mentah telemetri dan inventaris dalam format CSV Spreadsheet berstandar RFC 4180.

---

## 4.2 Kebutuhan Non-Fungsional (Non-Functional Requirements)

### A. Metrik Performa (Performance Benchmarks):

1. **API Response Time:** Waktu respon untuk permintaan pembacaan data (`GET /api/assets`, `GET /api/incidents`) tidak boleh melampaui **150 ms** pada kondisi beban normal dan **300 ms** pada beban puncak (*95th percentile*).
2. **Monitoring Cycle Throughput:** Eksekusi polling paralel terhadap 20 node perangkat wajib rampung secara sempurna dalam waktu $\le \mathbf{3.5\text{ detik}}$ per siklus.

3. **Frontend Rendering Latency:** Waktu render antarmuka web dasbor dari pembacaan memori lokal hingga selesai ditampilkan di layar (*First Meaningful Paint*) harus tercapai dalam waktu $\le \mathbf{1.2\text{ detik}}$ pada peramban Google Chrome desktop.
4. **Memory Footprint:** Penggunaan memori resident set size (RSS) proses backend Node.js tidak boleh melampaui **180 MB** dalam kondisi operasional stabil selama 30 hari terus menerus (*zero memory leak*).

### B. Metrik Keamanan (Security Constraints):

1. **Pencegahan Brute-Force:** Rute `POST /api/auth/login` wajib dibatasi maksimal **10 permintaan per menit per alamat IP**. Pelanggaran ambang batas langsung diganjar kode HTTP 429.
2. **Mitigasi SQL Injection:** Tidak ada query SQL yang dibangun melalui konkatenasi string mentah. Seluruh query wajib menggunakan fitur *Prepared Statements* dan sanitasi variabel melalui driver `mysql2`.
3. **Mitigasi Cross-Site Scripting (XSS):** Seluruh input teks dari form pengguna disanitasi menggunakan fungsi penyaring entitas HTML sebelum dirender ke DOM peramban.
4. **Kebijakan CORS & Header Keamanan:** Server hanya mengizinkan origin resmi korporat (`https://t-aims.teknolab.biz.id`). Header keamanan HTTP wajib menyertakan `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, dan `Referrer-Policy: strict-origin-when-cross-origin`.

### C. Skalabilitas & Kompatibilitas (Scalability & Compatibility):

1. **Skalabilitas Vertikal Perangkat:** Skema basis data dan arsitektur sweeper harus mampu menampung ekspansi hingga 100 node perangkat pemantauan tanpa perlu mengubah struktur tabel fundamental.
2. **Ketersediaan Layanan (Uptime Target):** Platform menjamin rasio ketersediaan sistem $\ge \mathbf{99.85\%}$ per bulan operasional di server produksi.
3. **Kompatibilitas Lintas Platform:** Antarmuka sistem terverifikasi berfungsi sempurna tanpa distorsi tata letak pada resolusi:
   - **Desktop Layar Lebar:** $1920\times1080$ px (Full HD) dan $1366\times768$ px (Laptop Standar).
   - **Tablet Operasional:** $1024\times768$ px (Apple iPad / Samsung Galaxy Tab).
   - **Ponsel Pintar Responsif:** $375\times812$ px (Apple iPhone) hingga $412\times915$ px (Android Modern).

---

# 5. BLUEPRINT INSTALASI, DEPLOYMENT, & MAINTENANCE

---

## 5.1 Prasyarat Lingkungan (Prerequisites Environment)

Sebelum memulai instalasi sistem, pastikan perangkat keras dan lingkungan perangkat lunak memenuhi kriteria minimum:

- **Sistem Operasi Server / Dev:** Ubuntu 22.04 LTS (Direkomendasikan), Debian 12, macOS Sonoma+, atau Windows 11 (disertai PowerShell 7+ / WSL2).
- **Runtime Node.js:** Node.js versi `v20.18.0 LTS` atau `v20.x` (minimal `v18.17.0 LTS`).
- **Package Manager:** `npm` versi `v10.x` atau lebih baru.
- **Basis Data:** MySQL Server versi `8.0.36+` atau MariaDB versi `10.11+`.
- **Process Manager:** `pm2` versi `v5.3+` (untuk lingkungan hosting produksi).
- **Web Server:** Nginx versi `1.24+` dengan modul SSL dan HTTP/2 terpasang.
- **Alat Jaringan:** Akses utilitas sistem `ping` (ICMP echo) yang tidak diblokir oleh kebijakan keamanan *SELinux* atau *AppArmor*.

---

## 5.2 Panduan Instalasi Lokal Langkah-demi-Langkah (Step-by-Step Local Setup)

### 1. Kloning Repositori Sumber

Buka terminal dan unduh repositori resmi tim dari GitHub:

```
git clone https://github.com/T-AIMS/T-AIMS.git
cd T-AIMS
```

### 2. Konfigurasi Basis Data MySQL

Masuk ke antarmuka baris perintah MySQL lokal:

```
mysql -u root -p
```

Eksekusi perintah SQL untuk menyiapkan basis data baru:

```
CREATE DATABASE taims_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'taims_user'@'localhost' IDENTIFIED BY 'PasswordKuatTaims2026!';
GRANT ALL PRIVILEGES ON taims_db.* TO 'taims_user'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Impor skema struktur tabel dan data master awal inventaris:

```
mysql -u taims_user -p taims_db < taims-backend/database/schema.sql
mysql -u taims_user -p taims_db < taims-backend/database/seeders.sql
```

### 3. Instalasi Dependensi Backend

Pindah ke direktori backend dan jalankan instalasi paket dependensi:

```
cd taims-backend
npm install
```

### 4. Konfigurasi Variabel Lingkungan ( `.env` )

Salin templat konfigurasi dan sesuaikan nilai parameter lingkungan lokal:

```
cp .env.example .env
nano .env
```

Pastikan isi berkas `.env` memuat konfigurasi yang presisi:

```
# ========================================
# KONFIGURASI SERVER RUNTIME T-AIMS
# ========================================
PORT=5000
NODE_ENV=development
```

```
# ==========================================
# KONEKSI BASIS DATA MYSQL
# ==========================================
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=taims_user
DB_PASSWORD=PasswordKuatTaims2026!
DB_NAME=taims_db
DB_CONNECTION_LIMIT=10

# ==========================================
# KEAMANAN OTENTIKASI JWT & BCRYPT
# ==========================================
JWT_SECRET=rahasia_otentikasi_jwt_taims_development_2026_super_aman
JWT_EXPIRES_IN=24h

# ==========================================
# MESIN PEMANTAU JARINGAN (MONITORING SWEEPER)
# ==========================================
PING_INTERVAL_MS=30000
PING_TIMEOUT_SECONDS=2
FLAPPING_COOLDOWN_SECONDS=120

# ==========================================
# INTEGRASI WEBHOOK & NOTIFIKASI
# ==========================================
N8N_AI_WEBHOOK=https://n8n.teknolab.biz.id/webhook/taims-ai-assistant
TELEGRAM_BOT_TOKEN=token_bot_telegram_resmi_anda
TELEGRAM_CHAT_ID=-100123456789
```

### 5. Menjalankan Server Backend

Jalankan backend server:

```
# Mode pengembangan dengan auto-reload:
npm run dev

# Atau mode standar:
node src/server.js
```

*Output verifikasi sukses pada terminal:*

```
[T-AIMS ENGINE] Server aktif mendengarkan koneksi pada port 5000.
[DATABASE] Kolam koneksi MySQL ke 'taims_db' berhasil diinisialisasi (Pool L
[MONITORING] Daemon pemantau aktif: Memeriksa 20 perangkat (Interval: 30s).
```

### 6. Menjalankan Server Frontend

Buka tab terminal baru di direktori root `T-AIMS`, lalu jalankan file server lokal untuk antarmuka klien:

```
# Menggunakan npx serve bawaan Node:
npx serve frontend -p 5500
```

Buka peramban web dan akses alamat: `http://localhost:5500`
Kredensial login bawaan:

- **Email:** `admin@taims.id`
- **Kata Sandi:** `AdminNOC2026!`

---

## 5.3 Strategi Deployment Produksi, Rotasi Log, & Pemeliharaan (Maintenance Blueprint)

### A. Alur Rilis Produksi Tanpa Downtime (*Zero-Downtime Deployment*):

1. **Akses Server Produksi:** Masuk via SSH terproteksi kunci kriptografis:

   ```
   ssh -i ~/.ssh/teknolab_vps_rsa root@server.teknolab.biz.id -p 22
   ```

2. **Pindah ke Direktori Proyek Produksi:**

   ```
   cd /home/taims-prod/htdocs/t-aims.teknolab.biz.id
   ```

3. **Tarik Versi Stabil Terakhir dari Branch Main:**

   ```
   git fetch origin main
   git reset --hard origin/main
   ```

4. **Instalasi Dependensi Bersih (Clean Production Dependency):**

```
cd taims-backend
npm ci --omit=dev
```

5. **Reload Proses Daemon Menggunakan PM2:**

```
pm2 reload taims-backend --update-env
```

*Perintah `reload` memastikan instance lama tetap melayani request pengguna yang sedang aktif hingga proses baru siap melayani trafik baru secara sempurna (Zero-Downtime).*

### B. Konfigurasi Nginx Virtual Host (CloudPanel Reverse Proxy):

Pastikan blok Nginx `/etc/nginx/sites-available/t-aims.teknolab.biz.id` terkonfigurasi dengan penanganan kompresi dan WebSocket upgrade:

```
server {
    listen 443 ssl http2;
    server_name t-aims.teknolab.biz.id;

    root /home/taims-prod/htdocs/t-aims.teknolab.biz.id/frontend;
    index index.html;

    # Penanganan Berkas Statis Klien SPA
    location / {
        try_files $uri $uri/ /index.html;
        expires 7d;
        add_header Cache-Control "public, no-transform";
    }

    # Reverse Proxy Menuju REST API Node.js
    location /api/ {
        proxy_pass http://127.0.0.1:5000/api/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 90;
        proxy_connect_timeout 90;
    }

    # Penanganan File Log Spesifik
    access_log /var/log/nginx/taims_access.log combined buffer=64k flush=5m;
    error_log /var/log/nginx/taims_error.log warn;

    # Konfigurasi Sertifikat SSL Otomatis Let's Encrypt
    ssl_certificate /etc/letsencrypt/live/t-aims.teknolab.biz.id/fullchain.p
    ssl_certificate_key /etc/letsencrypt/live/t-aims.teknolab.biz.id/privkey
}
```

### C. Strategi Pencatatan Log Sistem & Penanganan Crash (Logging & Monitoring):

1. **Pemisahan Aliran Log PM2:**
   Proses backend dikonfigurasi mencatat output standar (`stdout`) dan error standar (`stderr`) ke berkas terpisah:
   - **Info Log:** `/var/log/taims/taims-out.log` (pencatatan siklus monitoring dan autentikasi sukses).
   - **Error Log:** `/var/log/taims/taims-err.log` (pencatatan unhandled exception, error koneksi database, dan timeout webhook).

2. **Rotasi Log Otomatis (Logrotate):**
   Untuk mencegah pembengkakan ukuran harddisk server, pasang modul `pm2-logrotate`:

   ```
   pm2 install pm2-logrotate
   pm2 set pm2-logrotate:max_size 50M
   pm2 set pm2-logrotate:retain 14
   pm2 set pm2-logrotate:compress true
   ```

   *Konfigurasi di atas membatasi ukuran maksimal satu berkas log sebesar 50 Megabyte, mengompresi log lama dengan gzip, dan menyimpan riwayat maksimal selama 14 hari terakhir.*

3. **Crash Recovery & Auto-Restart:**
   Jika terjadi kondisi fatal yang memicu proses crash (*Uncaught Exception*), daemon PM2 secara otomatis menghidupkan kembali proses dalam waktu $< 1\text{ detik}$, serta

mengeksekusi script notifikasi darurat untuk memberi tahu Lead Architect via webhook darurat.

---

**DOKUMEN ARSITEKTUR & TEKNIS INI DISUSUN, DIVALIDASI & DITANDATANGANI SECARA RESMI OLEH:**

**Reza Tri Wardana**
*Principal Software Architect & Lead QA Engineer*
*PT Teknolab Caraka Internasional*
*Lisensi Rilis: Production Golden Master v2.5.0*